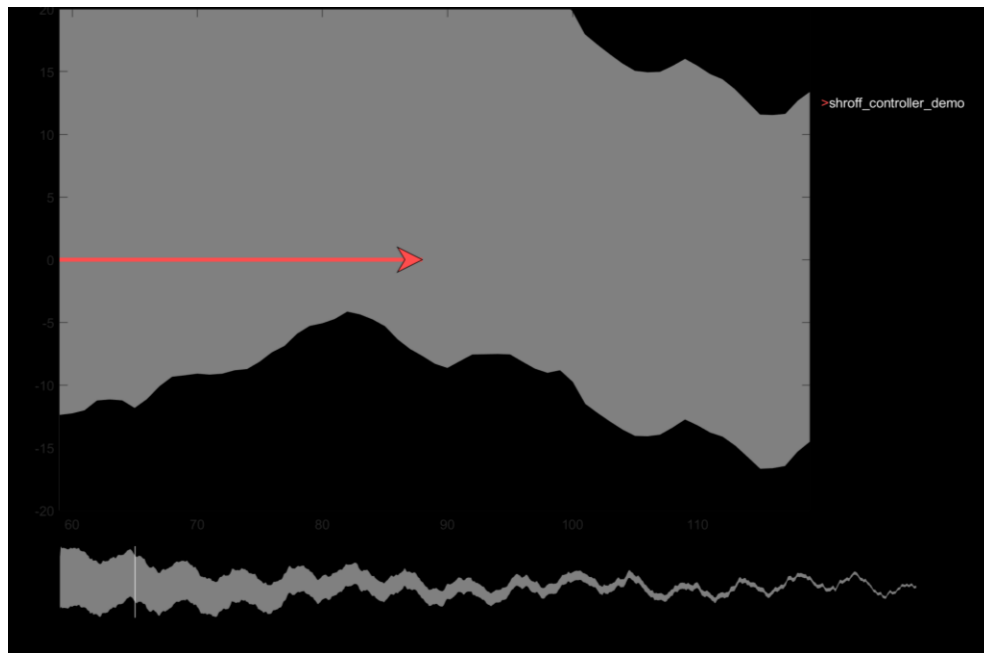


Homework 1: Cave Flyer



Instructions:

- 1) Run MATLAB using one of these methods:
 - a) Download and install MATLAB locally (<https://www.cmu.edu/computing/software/all/matlab/>).
 - b) Run MATLAB online through a browser (<https://www.mathworks.com/products/matlab-online.html>).
 - c) Run MATLAB through a Virtual Andrew session (<https://www.cmu.edu/computing/services/endpoint/software/managed-desktops/how-to/virtual-andrew.html>).
- 2) Download the file `CaveFly.zip` from the course files on Canvas, and unzip
- 3) Open MATLAB and navigate to the file. Add the folder containing the file to the MATLAB path (On the toolbar, select Environment | Set Path).
- 4) Open the latest version of `CaveFly.m`.
 - a) Set the value of `M.controller_dir` to point to the folder with your controllers
 - b) Read the other fields of `M` and understand the other modifiable parameters

Questions:

1) Your task is to design a controller for the system. Find the controller file with function

```
function F = shroff_controller(y, vy, ytop, ybot)
```

```
% Goal - use the input variables to calculate a desired output force F.
```

```
% ytop: a 1 x 21 vector with positions of the top wall. ytop(21) is the furthest from the ship.
```

```
% ybot: a 1 x 21 vector with positions of the bottom wall. ybot(21) is the furthest from the ship.
```

```
% y: Current position of the ship
```

```
% vy: Current velocity of the ship
```

```
% F: Output force to apply in the vertical direction (can be  $-1 \leq F \leq 1$ , code will enforce this)
```

Read and understand all of the inputs and output of the function.

- 2) In words, describe how you would avoid hitting walls of the cave. Hint: You could say something like if you get within x units of either wall, set F to push the ship the other way. An alternative scheme would be to try to stay in the middle of the path.
- 3) Convert your scheme to mathematical equations. Remember, you can only work with the input values in the function `F = yourname_controller(ytop, ybot, y, vy)`. Add code to the function representing these equations. Make sure to rename the .m file with your new function name `yourname_controller`.
- 4) Run the code with your controller. How well did you do? Describe your process for systematically tuning the controller (for example, how did you use educated guesses to tune the strength of forces?). Try a different control scheme if needed.
- 5) Run levels [1, 2, 4, 8, 16, ..., 2048] by changing `M.levels`. Record the impact time t for each level and take the average. Report the individual times on the class spreadsheet (https://docs.google.com/spreadsheets/d/185q_onHT4oSuj3FxN-12PGQVs_D_wGWYxbHLIN0gWTM/edit?usp=sharing).
- 6) In a few sentences describe important factors that were not taken into account in this simulation. Hint: think about sensor noise, delay in changing the thrust, etc. Could your controller be used on different ships? If the mass of a ship was twice as much, or the cave wall shapes were very different (more jagged for example), do you think your controller would have to be adjusted?
- 7) Submit answers to these questions as well as controller code on Canvas.

Optional Upload:

Upload your controller file here. We will demo and discuss some of these after the assignment is due.

<https://forms.gle/pNfxWdQJNpRe6MuE6>

Optional Bonus:

Add a loop in the code to quickly test your controller on levels [1, 2, 4, 8, ..., 2048]. Save the times in a vector, and take the average. Try to optimize the average time until collision by

changing parameter values in your code. Show that you have found a local minima by demonstrating that by either increasing or decreasing one of the parameters, the average time decreases.